Async e Await

Como o JavaScript espera sem travar

Sumário

ENTENDENDO O PROBLEMA

Tarefas que demoram

Uma coisa de cada vez

Promises

A promessa de um resultado futuro

NA PRÁTICA

async e await

“pode demorar” e “vou esperar aqui”

Event loop passo a passo

Erros, paralelo e Express

try/catch · Promise.all · rotas async

Algumas tarefas demoram

Quando o código depende de algo **fora dele**, o resultado não chega na hora:

BANCO DE DADOS

```
prisma.livro.findMany()
```

INTERNET

```
fetch("https://api...")
```

ARQUIVOS

```
fs.readFile("dados.txt")
```

TEMPO

```
setTimeout(..., 1000)
```

Enquanto isso, o programa não pode ficar parado.

Um garçom só

O JavaScript faz **uma coisa de cada vez**. Pense em um restaurante com um único garçom:

Esperando parado

O garçom anota o pedido e fica na cozinha até o prato ficar pronto. Ninguém mais é atendido.

Esperando sem travar

O garçom anota o pedido, atende as outras mesas e volta quando a cozinha avisa que o prato está pronto.

O **async/await** é a forma de escrever o segundo garçom.

Promise

Uma tarefa que demora devolve uma **promessa de resultado**. Ela está sempre em um de três estados:

`pending`

Pendente. A tarefa ainda está em andamento.

`fulfilled`

Resolvida. Deu certo e o valor está disponível.

`rejected`

Rejeitada. Algo deu errado e temos um erro.

O **async/await** existe para trabalhar com Promises de um jeito simples de ler.

Sem esperar, o que acontece?

01-sem-await.js

```
async function buscarLivro() {
  await esperar(1000);
  return { id: 1, titulo: "Dom Casmurro" };
}



const livro = buscarLivro();
console.log(livro);
```

RESULTADO

```
Promise { <pending> }
```

A variável **livro** não guarda o livro. Guarda a promessa de um livro que ainda não chegou.

async

“Essa função pode demorar.”

```
async function buscarLivro() {
  return { id: 1 };
}
```

Avisa que a função pode levar tempo

Sempre devolve uma **Promise**

Libera o uso do **await** dentro dela

await

"Vou ficar aqui até terminar."

```
const livro = await buscarLivro();
console.log(livro);
```

- Pausa a função até a Promise terminar
- Entrega o **valor**, não a Promise
- Só funciona dentro de uma função **async**
- Pausa só a função. O resto do programa continua

Juntos

```
async function listarLivros() {
  const livros = await prisma.livro.findMany();
  console.log(livros);
}
```

async

Fica na **função**: “essa função pode demorar”.

await

Fica na **chamada**: “vou ficar aqui até terminar”.

Fingindo uma tarefa lenta

Para treinar, vamos criar uma função que **espera alguns milissegundos**, como se fosse um banco de dados lento.

esperar.js

```
export function esperar(ms) {
  return new Promise((resolve) ⇒
    setTimeout(resolve, ms)
  );
}
```

02-com-await.js

```
const livro = await buscarLivro();
console.log(livro);
```

DEPOIS DE 1 SEGUNDO

```
{ id: 1, titulo: 'Dom Casmurro' }
```

Event loop: as peças

É assim que o JavaScript espera **sem travar**. Quatro peças trabalham juntas:

01

Pilha

Onde o código roda, uma coisa por vez.

02

Node

Cuida do trabalho demorado: timers, banco, rede.

03

Fila

Guarda o que já terminou e espera a vez de rodar.

04

Event loop

Vigia a pilha. Se estiver vazia, leva o próximo da fila.

Ele repete isso o tempo todo, por isso se chama **loop**.

Event loop na prática

passo 1 de 5

```
  async function main() {

>   console.log("1 - começou");
    await esperar(1000);
    console.log("3 - terminou");
  }


  main();
  console.log("2 - fora da função");
```

PILHA

main()

NODE

vazio

FILA

vazia

SAÍDA

```
1 - começou
```

A **main()** foi chamada: entra na pilha e imprime a primeira linha.

Event loop na prática

passo 2 de 5

```
  async function main() {
    console.log("1 - começou");

>   await esperar(1000);
    console.log("3 - terminou");
  }


  main();
  console.log("2 - fora da função");
```

PILHA

vazia (a main pausou)

NODE

`timer: 1000 ms`

FILA

vazia

SAÍDA

`1 - começou`

O **await** entrega o timer ao Node e tira a main da pilha. A função fica pausada e a pilha fica livre.

Event loop na prática

passo 3 de 5

```
  async function main() {
    console.log("1 - começou");
    await esperar(1000);
    console.log("3 - terminou");
  }


  main();

> console.log("2 - fora da função");
```

PILHA

`console.log()`

NODE

`timer: 1000 ms`

FILA

vazia

SAÍDA

```
1 - começou
2 - fora da função
```

Com a pilha livre, o resto do programa segue: a linha de fora roda **sem esperar o timer**.

Event loop na prática

passo 4 de 5

```
  async function main() {
    console.log("1 - começou");

>   await esperar(1000);
    console.log("3 - terminou");
  }


  main();
  console.log("2 - fora da função");
```

PILHA

vazia

NODE

timer terminou

FILA

continuar main()

SAÍDA

1 - começou
2 - fora da função

Passou 1 segundo. O timer terminou e “continuar a main” entrou na **fila**. O event loop vê a pilha vazia.

Event loop na prática

passo 5 de 5

```
  async function main() {
    console.log("1 - começou");
    await esperar(1000);

>   console.log("3 - terminou");
  }


  main();
  console.log("2 - fora da função");
```

PILHA

`main() retomada`

NODE

vazio

FILA

vazia

SAÍDA

```
1 - começou
2 - fora da função
3 - terminou
```

O event loop move o item da fila para a pilha. A main **continua de onde parou** e imprime o último passo.

E se der erro?

Uma Promise rejeitada vira um erro no **await**. Quem captura é o **try/catch**.

04-erro.js

```
try {
  const livro = await buscarLivro(99);
  console.log(livro);
} catch (erro) {
  console.log("Deu erro:", erro.message);
}
```

RESULTADO

```
Deu erro: Livro não encontrado
```

Sem o try/catch, o erro derrubaria o programa. É o mesmo conceito da aula de Try-Catch, agora com await.

Um de cada vez ou todos juntos

Cada **await** espera terminar antes de seguir. Para tarefas independentes, dá para **começar todas ao mesmo tempo**.

SEQUENCIAL

```
await tarefa("A");

await tarefa("B");

await tarefa("C");
```

≈ 3 segundos

PROMISE.ALL

```
await Promise.all([
  tarefa("A"), tarefa("B"),
  tarefa("C"),
]);
```

≈ 1 segundo

Na prática: rota com Prisma

```
app.get("/livros", async (req, res) ⇒ {
  try {
    const livros = await prisma.livro.findMany();
    res.json(livros);
  } catch (erro) {
    res.status(500).json({ erro: "Erro no banco" });
  }
});
```

`async`
na função da rota

`await`
na consulta ao banco

`try/catch`
para responder se o banco falhar

Enquanto uma consulta espera o banco, o servidor **atende outras requisições**.

Três erros comuns

01

Esquecer o await

A variável guarda uma Promise pendente, não o valor.

```
const x = buscarLivro();
```

02

await sem async

Dá erro de sintaxe dentro de uma função comum.

```
function f() { await g(); }
```

03

async dentro do forEach

O forEach não espera. Use for…of.

```
for (const id of ids)
```

Os três estão em exemplos que dá para rodar: **01-sem-await.js** e **06-for-await.js**.

RESUMO

`async`

“Essa função pode demorar.”

`await`

“Vou ficar aqui até terminar.”

Por baixo, o event loop pausa a função, libera a pilha e retoma quando o resultado chega.

Hora de praticar

01

Rode **01-sem-await.js** e **02-com-await.js**. O que muda na saída?

02

Em **03-event-loop.js**, apague a linha do await. Em que ordem as mensagens aparecem?

03

Em **04-erro.js**, troque o 99 por 1. O que aparece agora?

04

Em **05-paralelo.js**, adicione uma quarta tarefa. Quanto tempo cada versão leva?

Desafio

Na sua API de livros, crie a rota **GET /livros** com **async**, **await** e **try/catch**.